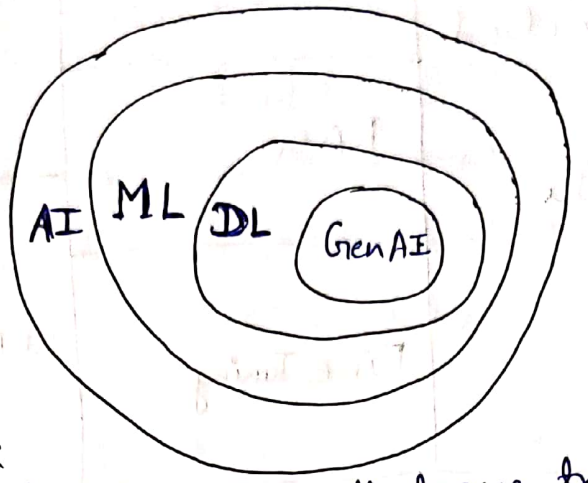
Introduction to Generative AI

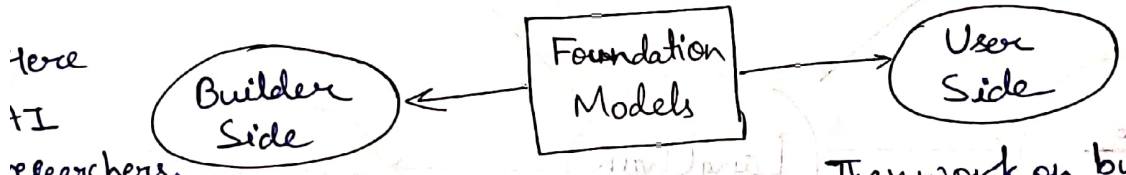
What is Gen AI? Generative AI is a type of artificial intelligence that creates new content - such as text, images, music, code - by learning patterns from existing data mimicking human activity.



Foundation Models

These are large deep learning models that are trained on very large amount of data. These are generalized models that serve as "base" upon which many other applications can be built. These can be fine tuned for tasks like translation, summarization, classification, image generation, etc.

Examples ⇒ GPT, LLaMA, Gemini, BERT, Stable Diffusion, etc.



Here, Software Developers, End Users exists.

Here AI researchers, Data Scientists, ML Engineers, etc. exist. They work on creating the foundation models.

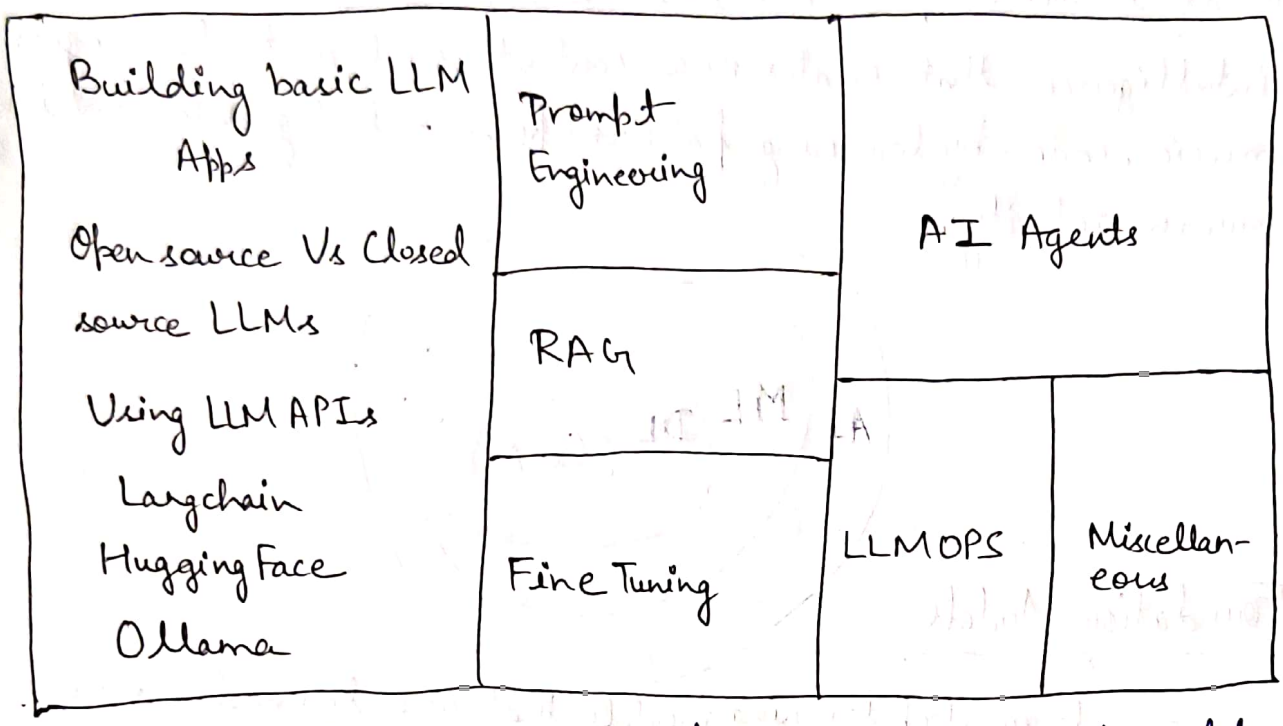
They work on building application softwares using the available foundation models released via big tech companies.

Builder's Perspective

Transformer architecture	Pretraining Training Objectives Tokenization strategies Training strategies Handling Challenges	Fine tuning Task specific tuning Instruction tuning Continual Pretraining	RLHF PEFT
Types of transformer Encoder Only (BERT) Decoder Only (GPT) Encoder & Decoder Based (T5)	Optimization Training Model Compression Inference	Evaluation	Deployment

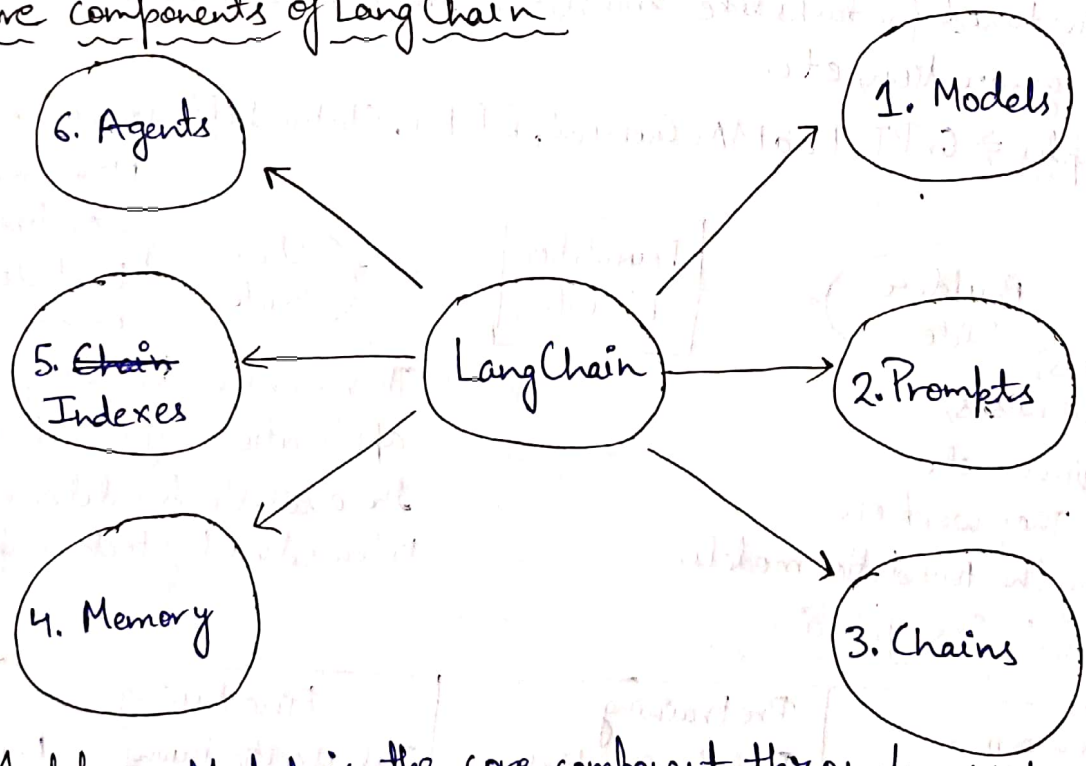
Scanned with OKEN Scanner

User's Perspective



LangChain LangChain is an open-source framework used to build LLM-powered applications.

Core components of LangChain



1. Models Model is the core component through which we interact with AI models. LangChain provides a standardized framework using which we can use any of the models provided by different organizations like OpenAI, Claude, Gemini, etc.

Scanned with OKEN Scanner

Without LangChain, switching providers rewriting code because the request/response format differs for each provider. However with the help of LangChain, we can switch LLM providers by changing just the package name, while the rest of the code remains consistent.

Types of models in LangChain

1. Language Models (LLMs): These take text input and provide text output and are mostly used for chatbots and agents.

2. Embedding Models: These take text input and provide their corresponding vectors (numeric representation) as output. These are mostly used for semantic search.

2. Prompts

The input that is passed to the LLM is called prompt. The output generated by LLMs is highly sensitive to prompts. LangChain provides different ways in which we can provide prompts.

(a) Dynamic and Reusable Prompts: Here, we can create templates with placeholders (eg. Summarize [Topic] in [Emotion] tone). We can swap these placeholders dynamically.

(b) Role-based Prompts: We can even instruct the model as per its persona. For Eg. A system prompt and a User prompt.

(c) Few-shot Prompts: Here, we provide the model with a few examples of inputs and desired outputs (eg. showing examples of "billing issues" vs "technical issues") before asking it to classify a new query.

3. Chains

Chains are the components that allows us to create pipelines in LangChain. Using chains the output of one stage can become input to another stage without writing any manual code to handle the transfer. Different types of chains are listed below:

(a) Sequential Chain ⇒ Linear processing (one after another).

(b) Parallel Chain ⇒ Sending the same input to multiple LLMs simultaneously (e.g. One LLM writes a report, another checks facts) and combining the result.

Scanned with OKEN Scanner

(c) Conditional Chains ⇒ In this the processing happens based on logic (eg. let's say if a feedback is positive → Say "Thank You"; if feedback is negative → email support team).

4. Indexes This component connects our application to external knowledge sources like PDFs, websites or databases. It has four sub-components.

(a) Document Loaders ⇒ These are responsible for bringing data from a source (e.g. Google Drive, Local Files, etc.)

(b) Text Splitters ⇒ It breaks large documents into smaller chunks (pages, paragraphs or chapters).

(c) Vector Stores / Databases ⇒ It stores the embedding vectors (numerical representations) of the text chunks to enable semantic search.

(d) Retrievers ⇒ It takes a user query, converts it to a vector, searches the database for relevant chunks, and sends them back to the LLM.

5. Memory LLM API calls are "stateless". The model requests every request independently and does not remember previous interactions. Thus, the Memory component adds state/history to conversations. These also have various types :-

(a) Conversational Buffer Memory ⇒ It stores the entire conversation history. (Good for context, but can become expensive/large).

(b) Conversational Buffer Window Memory ⇒ It stores the last N interactions. (e.g. Last 10 messages).

(c) Summarized Based Memory ⇒ It creates a summary of the previous conversation to send to LLM, saving token space.

(d) Custom Memory ⇒ It is used for storing user facts or preferences.

Scanned with OKEN Scanner

6. Agents

It is an advanced component that allows LLMs to perform actions. Key Characteristics ⇒

(a) Reasoning Capacity ⇒ The ability to break down a problem (step-by-step) (e.g. Chain of Thought).

(b) Access to Tools ⇒ The agent is given access to external tools, using which it can perform several actions.

Models
- → Language models
  - → LLMs (Legacy/General Purpose)
  - → Chat Models (Modern/Conversational)
- → Embedding models

Ⓐ LLMs ⇒ It takes a string as an input and generates a string as an output. These are mostly used for text completion kind of jobs.

Ⓑ Chat Models ⇒ It can take a list of messages (System/User/AI) and returns a message object. It supports varying roles (e.g. defining "You are a doctor"), conversation history and context awareness.

⇔ Key Parameters

↳ Temperature: It controls the creativity of the answers generated by the LLM model. Its value generally ranges between (0 & 2).

→ 0-0.3 (Deterministic) ⇒ If we send the same input everytime, after setting temperature = 0, it gives back the same answer.

→ > 1 (Stochastic/Creative) ⇒ If we set it high (eg. 1.5), sending the same inputs will result in different outputs every time.

Usage Rule ⇒ Use low temperature for apps requiring consistency (e.g. math, extraction). Use high temperature where variety is desired (e.g. brainstorming, poetry).

↳ Max Tokens: It limits the number of words in the generated output to control costs.

Scanned with OKEN Scanner